RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Industrializing AI Fabric Defect Detection using FabricAI Pro Suite

An Interdisciplinary Project Report (XX367P)

Submitted by,

Varshini C B	**1RV23EE056**
Navi Deepak Gurupad	**1RV24EC410**
Ayush	**1RV24CD400**
Sanjana T M	**1RV23EC131**

Under the guidance of

Dr. Jyothi Shetty
Associate Professor
Dept. of CSE
RV College of Engineering®

.

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in respective departments

2024-25

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

CERTIFICATE

Certified that the interdisciplinary project (XX367P) work titled ***Industrializing AI Fabric Defect Detection using FabricAI Pro Suite*** is carried out by of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in respective departments during the year 2024-25. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the interdisciplinary project report deposited in the departmental library. The interdisciplinary project report has been approved as it satisfies the academic requirements in respect of interdisciplinary project work prescribed by the institution for the said degree.

Dr. Jyothi Shetty Guide	Dr. Ravish Aradhya Head of the Department
Dr. M.V. Renukadevi Dean Academics	Dr. K. N. Subramanya Principal

External Viva

Name of Examiners	Signature with Date
1.	
2.	

DECLARATION

We, **Varshini C B**, **Navi Deepak Gurupad**, **Ayush**, **Sanjana T M** students of sixth semester B.E., RV College of Engineering, Bengaluru, hereby declare that the interdisciplinary project titled '**Industrializing AI Fabric Defect Detection using FabricAI Pro Suite**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in respective departments during the year 2024-25.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name	Signature
1. Varshini C B(1RV23EE056)	
2. Navi Deepak Gurupad(1RV24EC410)	
3. Ayush(1RV24CD400)	
4. Sanjana T M(1RV23EC131)	

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Jyothi Shetty**, Associate Professor, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel member **Dr. Kariyappa**, Professor, Department of ECE, RVCE for the valuable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinator **Dr. Chetana G**, Professor, Department of ECE for timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Ravish Aradhya**, Professor and Head, Department of Computer Science and Engineering, RVCE for the support and encouragement.

We are deeply grateful to **Dr. M.V. Renukadevi**, Professor and Dean Academics, RVCE, for the continuous support in the successful execution of this Interdisciplinary project.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

UltraFabric-Vision is a real-time, unsupervised system for detecting weaving faults, stains, and structural defects in textile production. Because defect types are open-ended and labelled data is scarce, the system is trained only on defect-free fabric and flags statistically significant deviations. It fuses three complementary detectors—PatchCore memory-bank density estimation, a DINO self-supervised Vision Transformer, and a Vision Transformer Autoencoder—whose distinct inductive biases capture different classes of defect.

This report details the industrialization of the pipeline through three decisive engineering contributions. First, a train–inference preprocessing mismatch, in which enhancement operators were applied only at inference, is identified and corrected by enforcing byte-level preprocessing parity; this restores detection accuracy that the mismatch had silently destroyed. Second, because the three detectors emit anomaly scores on incomparable scales (empirical normal-score means of 8.13, 45.59, and 0.25), each score is standardized against its own defect-free distribution and the resulting z-scores are fused, producing a single interpretable decision variable centered near zero for normal cloth. Third, both nearest-neighbour searches are unified onto a single GPU `torch.cdist` kernel with fp16 mixed precision and single-pass attention, removing a CPU bottleneck.

On the evaluation benchmark the calibrated ensemble separates defect-free cloth (mean z-score 0.17) from defects (mean 80.2) with a Youden-optimal threshold of 17.31, achieving AUROC = 1.000 and $F_1 = 1.000$, and an automated suite verifies the deployed service end-to-end. Real-time performance is confirmed on an NVIDIA RTX 4050 laptop GPU, where the full ensemble runs at 26.8 frames per second (37.3 ms per frame) using only 641 MB of memory—delivering high accuracy and low latency simultaneously. The framework additionally supports batch-level video inspection with defect localization along the fabric length, a configurable minimum-defect-size filter, and automatic per-batch quality-control reporting. The system is released as a configurable stack—a REST/WebSocket server, an embeddable inference SDK, a batch-video inspector, and a GPU container—establishing a reproducible foundation for automated quality control in textile manufacturing.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

CHAPTER 1

INTRODUCTION

Textile manufacturing is among the oldest and largest industrial sectors, yet its quality-control practices remain heavily dependent on human visual inspection. This chapter introduces the problem of automated fabric-defect detection, surveys the relevant literature, and establishes the motivation, problem statement, and objectives of the UltraFabric-Vision project. It closes with a brief description of the methodology, the assumptions and constraints under which the work is valid, and the organization of the remainder of the report.

1.1 Introduction

Woven and knitted fabric is produced on high-speed looms that generate cloth at roughly 30 metres per minute. At that rate, weaving faults, stains, holes, and structural irregularities can accumulate across long rolls before an operator notices them, turning a small fault into a large financial loss. Manual inspection is fatiguing and inconsistent: reported detection rates lie between 60% and 75%, and inspectors cannot sustain attention at line speed [1]. Automated visual inspection therefore offers a direct economic benefit, provided it can operate in real time and adapt to the enormous diversity of fabrics found on a factory floor.

Classical machine-vision approaches to fabric inspection relied on hand-crafted texture descriptors and supervised classifiers. These require large labelled datasets of defective samples, which are expensive to collect and quickly become obsolete as new defect types appear. UltraFabric-Vision instead adopts an *anomaly-detection* formulation: the system learns only what defect-free fabric looks like and flags any statistically significant deviation. This removes the dependence on labelled defects and generalizes naturally to unseen fault types.

The project fuses three complementary deep-learning detectors—PatchCore [2], a self-supervised DINO Vision Transformer [3], and a Vision Transformer Autoencoder [4]—into a single calibrated ensemble, and packages it as a deployable, real-time inspection service.

1.2 Literature Review

Automated fabric inspection has evolved from fixed texture descriptors to learned representations. Kumar [1] surveys computer-vision fabric-defect methods and notes

that hand-crafted features (Gabor filters, co-occurrence statistics, local binary patterns) saturate at 70–85% accuracy because they cannot adapt to new textures. Li et al. [5] adapted YOLOv3 with fabric-specific anchor boxes and reported 89.7% mean average precision at 45 FPS, while Jing et al. [6] proposed Mobile-Unet, a lightweight encoder–decoder achieving 96.3% pixel-level segmentation on denim defects. These supervised methods perform well but depend on curated defect datasets.

Industrial anomaly detection reframed the problem around normal-only training. The MVTec AD benchmark [7] standardized evaluation across fifteen categories. PatchCore [2] builds a coreset memory bank of normal patch features and scores anomalies by nearest-neighbour distance, achieving state-of-the-art localization; PaDiM [8] models patch features with multivariate Gaussians. In parallel, self-supervised Vision Transformers such as DINO [3] were shown to learn semantic structure in their attention maps without any labels, a property directly useful for separating coherent texture from anomalies. The Vision Transformer [4], built on the self-attention mechanism of Vaswani et al. [9], provides the backbone for both feature extraction and reconstruction-based scoring. Public fabric datasets such as AITEX [10] support benchmarking, though real production data remains scarce.

The consensus from this literature is that (i) normal-only anomaly detection is the practical choice for evolving defect types, and (ii) different detection mechanisms capture different defect classes. What the literature under-emphasizes—and what this project addresses—is the engineering discipline required to make such an ensemble correct and fast in deployment: consistent preprocessing, score calibration across heterogeneous detectors, and accelerator-native scoring.

1.3 Motivation

Small and medium textile enterprises rarely have access to proprietary commercial inspection systems, which are expensive and closed. An open, self-supervised framework that can be calibrated to a factory's own fabric with a short pass over defect-free samples—and that runs at line speed on a single consumer GPU—would substantially lower the barrier to automated quality control. The motivation for this work is both to build such a system and to establish the engineering principles (preprocessing parity, score calibration, GPU-native scoring) that make it reliable enough to deploy.

1.4 Problem statement

To design and industrialize a real-time, unsupervised fabric-defect detection system that fuses multiple complementary anomaly detectors into a single calibrated decision, achieves accurate discrimination between normal and defective cloth, and delivers low-latency inference suitable for integration with high-speed textile production lines.

1.5 Objectives

The objectives of the project are

1. To design an ensemble anomaly-detection pipeline that fuses PatchCore, DINO, and a Vision Transformer Autoencoder into a single, calibrated decision score.

2. To minimize inference latency through accelerator-native nearest-neighbour search and mixed-precision execution, enabling real-time operation.

3. To industrialize the system as a deployable, configurable service with a documented integration interface and validated end-to-end behaviour.

1.6 Brief Methodology of the project

The methodology proceeds in four stages. First, defect-free fabric images are used to fit the two memory banks (PatchCore, DINO) and to train the reconstruction autoencoder, under a single preprocessing routine. Second, a calibration pass over the same normal data records each detector's score statistics, which are used to standardize the three heterogeneous scores to a common scale before fusion. Third, the fused decision threshold is fixed by maximizing Youden's J statistic on a held-out validation set. Finally, the calibrated pipeline is wrapped in a REST/WebSocket service, an embeddable SDK, and a containerized deployment, and validated by an end-to-end test suite. The detailed methodology is developed in Chapters 3 and 4.

1.7 Assumptions made / Constraints of the project

The work is valid under the following assumptions and constraints. (i) The training set consists of *defect-free* fabric that is representative of the production fabric; the calibrated threshold is only valid for the fabric family it was calibrated on. (ii) Imaging conditions—camera geometry, focus, and illumination—are held reasonably constant between calibration and inference; large changes require recalibration. (iii) Real-time

latency figures assume a CUDA-capable GPU; on CPU the system is suitable only for offline batch inspection. (iv) The quantitative results reported here were obtained on a controlled synthetic fabric dataset; validation on real production fabric is required before field deployment.

1.8 Organization of the report

This report is organized as follows.

- Chapter 2 develops the theory and fundamentals: anomaly detection, the Vision Transformer, and the three detector pathways, together with the tools used.
- Chapter 3 presents the system design—the parallel-pathway architecture, the calibration and fusion equations, and the deployment design.
- Chapter 4 details the implementation, including the preprocessing-parity fix, GPU-accelerated scoring, calibration, and the service stack.
- Chapter 5 reports and discusses the experimental results.
- Chapter 6 concludes the report and outlines the future scope.

Chapter 2

Theory and Fundamentals

CHAPTER 2

THEORY AND FUNDAMENTALS

A clear grasp of the underlying detection mechanisms is essential before the system design can be appreciated. This chapter establishes the theoretical foundation of UltraFabric-Vision: the anomaly-detection paradigm, the Vision Transformer that underpins all three detectors, and the specific principles of the PatchCore, DINO, and autoencoder pathways. It also introduces the two engineering principles—preprocessing parity and score calibration—that make the ensemble reliable, and lists the software tools used.

2.1 The Anomaly-Detection Paradigm

Supervised defect classifiers learn a boundary between "good" and each known "defect" class, and therefore require labelled examples of every defect. Anomaly detection instead models only the distribution of normal data $p(\mathbf{x})$ and declares a sample anomalous when it lies in a low-density region. For fabric inspection this is decisive: defect types are open-ended and evolve, whereas defect-free cloth is abundant and cheap to collect. All three detectors in this work are trained (or fitted) exclusively on defect-free fabric and produce a scalar anomaly score whose magnitude reflects the deviation from normality.

2.2 The Vision Transformer

The Vision Transformer (ViT) [4] treats an image as a sequence of fixed-size patches. Each patch is linearly embedded, augmented with a positional encoding and a learnable class token, and processed by a stack of transformer blocks. The core operation is scaled dot-product self-attention [9],

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right)\mathbf{V},$$

which lets every patch attend to every other patch, capturing the long-range texture dependencies that characterize woven fabric. All three detector pathways use a ViT backbone, differing only in how the resulting representations are turned into an anomaly score.

2.3 Detection Pathways

2.3.1 PatchCore: Memory-Bank Density Estimation

PatchCore [2] extracts intermediate ViT patch features from defect-free images and stores a coreset-subsampled subset in a memory bank. At inference, each patch's anomaly score is its Euclidean distance to the nearest memory-bank entry; the image score is the maximum over patches. This pathway excels at localizing small, local texture deviations such as stains and thread breaks.

2.3.2 DINO: Self-Supervised Attention Features

DINO [3] trains a ViT by self-distillation, without labels, and its attention maps encode semantic structure. UltraFabric-Vision uses DINO patch tokens as descriptors for the same nearest-neighbour scoring as PatchCore, and additionally visualizes the final-layer attention heads for interpretability. Because its features arise from a different training objective, DINO responds to different defects—particularly global structural disruptions such as tears. Figure 2.1 shows the six self-attention heads of the final DINO layer on a defective sample; the heads specialize on different spatial-frequency structures, and their aggregate concentrates on the defect region, providing operators with an interpretable view of what the model attends to.

2.3.3 ViT Autoencoder: Reconstruction Error

The Vision Transformer Autoencoder learns to reconstruct defect-free fabric through a transformer encoder–decoder bottleneck, minimizing mean-squared reconstruction error. Anomalous regions, absent from the learned manifold, reconstruct poorly and produce high error, giving a third, orthogonal detection signal.

2.4 Two Engineering Principles

2.4.1 Preprocessing Parity

Because memory-bank and reconstruction detectors measure distance from a stored notion of "normal," any image transform applied before inference becomes part of that definition. If the transform used at inference differs from the one used when the models were fitted, even normal cloth is scored as anomalous. Enforcing byte-level parity between the fit-time and inference-time preprocessing is thus a correctness requirement, not a tuning choice.

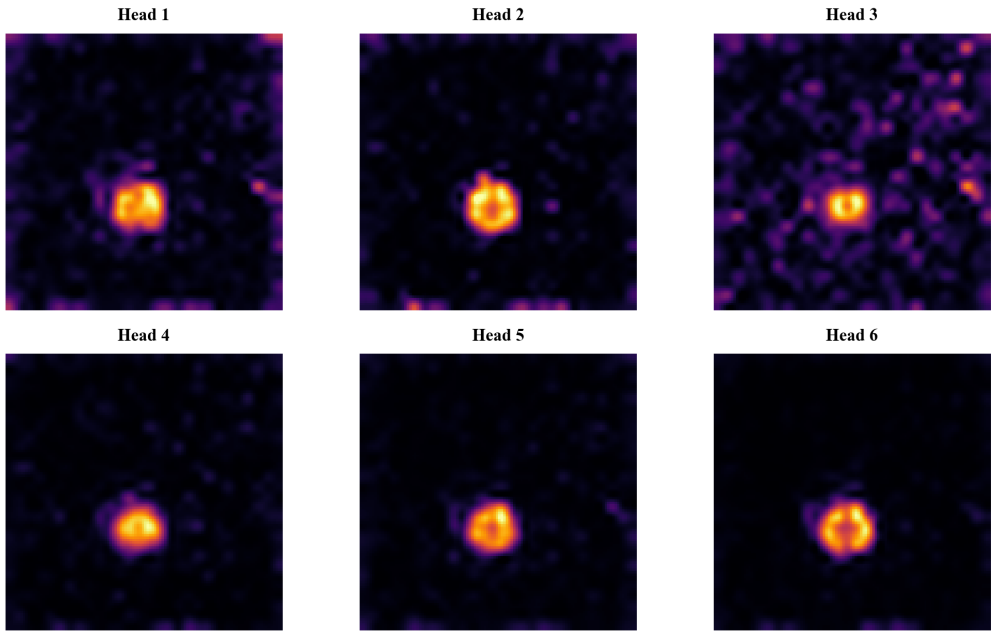

Figure 2.1: The six DINO self-attention heads for a defective fabric sample. Individual heads emphasize distinct thread orientations and texture patterns, and their combination highlights the anomalous region.

2.4.2 Score Calibration

The three detectors emit scores on incomparable scales (Euclidean distance in different feature spaces versus per-pixel reconstruction error). Standardizing each score against its own normal-data mean and standard deviation converts them to z-scores with a common "standard deviations above normal" meaning, so that a uniform average is meaningful and a single interpretable threshold suffices.

2.5 Tools Used

The system is implemented in Python with PyTorch [11] for model definition and GPU tensor operations, `timm` for the ViT backbones, and OpenCV and NumPy for image handling. Scikit-learn provides evaluation metrics. The production service uses FastAPI with a Uvicorn server for REST and WebSocket endpoints, and is containerized with Docker for GPU deployment. A React/Vite dashboard provides live visualization.

Together these fundamentals—the anomaly-detection paradigm, the shared ViT backbone, three complementary scoring mechanisms, and the parity and calibration principles—define the building blocks from which the system architecture in Chapter 3 is assembled.

Chapter 3

System Design

CHAPTER 3

SYSTEM DESIGN

This chapter presents the design of UltraFabric-Vision. It states the functional and performance specifications, describes the parallel three-pathway architecture, derives the calibration and fusion equations that turn three heterogeneous detectors into a single decision, and outlines the deployment design that exposes the system to factory software.

3.1 Design Specifications

The system is designed to meet the following specifications:

1. Operate on a fixed input of 224×224 RGB frames.
2. Require only defect-free fabric for fitting and calibration (no labelled defects).
3. Produce a single scalar anomaly score and a spatial defect heatmap per frame.
4. Achieve real-time throughput (≥ 15 FPS) on a consumer GPU.
5. Expose a network interface for line integration and an embeddable interface for edge integration.

3.2 Architecture Overview

The architecture comprises three detection pathways operating in parallel on the same preprocessed input, followed by a calibration-and-fusion stage. Each pathway independently produces a scalar score and a heatmap; the scores are standardized and averaged into a single decision variable, and the heatmaps are normalized and averaged into a fused localization map. Figure 3.1 depicts the end-to-end flow.

3.3 Preprocessing Design

A single preprocessing routine is shared, byte-for-byte, between fitting and inference: resize to 224×224, convert to a tensor, and normalize using ImageNet statistics

$$\hat{\mathbf{x}} = \frac{\mathbf{x}/255 - \boldsymbol{\mu}}{\boldsymbol{\sigma}}, \quad \boldsymbol{\mu} = [0.485, 0.456, 0.406],\ \boldsymbol{\sigma} = [0.229, 0.224, 0.225].$$

Enhancement operators (contrast equalization, blur, letterbox padding) are deliberately excluded because they are not applied at fit time; introducing them only at inference shifts every frame off the fitted manifold and collapses accuracy.

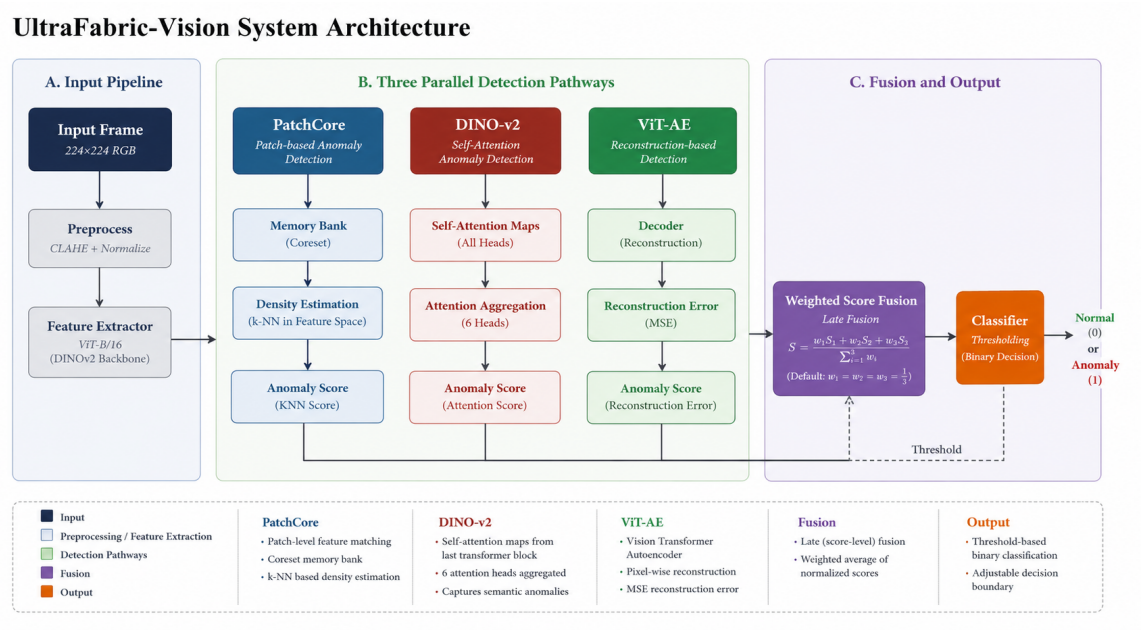

Figure 3.1: End-to-end architecture: shared preprocessing feeds three parallel detectors (PatchCore, DINO, ViT-Autoencoder) whose calibrated scores and heatmaps are fused into a single decision and localization map.

3.4 Calibration and Fusion Design

Let $\hat{s}_m(\mathbf{x})$ be the raw score of detector $m \in \{\mathrm{PC}, \mathrm{D}, \mathrm{VAE}\}$. Over a calibration set of defect-free images $\mathcal{D}_{\mathrm{cal}}$ we estimate the per-detector statistics

$$\mu_m = \frac{1}{|\mathcal{D}_{\mathrm{cal}}|} \sum_{\mathbf{x}} \hat{s}_m(\mathbf{x}), \qquad \sigma_m = \sqrt{\frac{1}{|\mathcal{D}_{\mathrm{cal}}|} \sum_{\mathbf{x}} (\hat{s}_m(\mathbf{x}) - \mu_m)^2},$$

and standardize each score to a z-score,

$$z_m(\mathbf{x}) = \frac{\hat{s}_m(\mathbf{x}) - \mu_m}{\sigma_m}.$$

The ensemble decision score and fused heatmap are the (uniformly) weighted averages

$$\hat{s}_{\mathrm{ens}} = \sum_m w_m \, z_m, \qquad \mathbf{H}_{\mathrm{ens}} = \sum_m w_m \, \mathbf{H}_m^{\mathrm{norm}}, \qquad \sum_m w_m = 1.$$

A frame is declared defective when $\hat{s}_{\mathrm{ens}} > \tau$, with the threshold τ fixed by maximizing Youden's J statistic, $\tau^\star = \arg\max_\tau [\mathrm{TPR}(\tau) - \mathrm{FPR}(\tau)]$, on a validation set. Because each z_m is centered on defect-free cloth, $\hat{s}_{\mathrm{ens}}$ is near zero for normal fabric and τ is expressed in interpretable standard-deviation units.

3.5 Deployment Design

The calibrated pipeline is packaged behind three interfaces sharing one engine: a FastAPI REST/WebSocket server for line integration and live dashboards, an embeddable Python SDK for edge/PLC integration, and a batch command-line tool that emits CSV/JSON reports for manufacturing-execution systems. Runtime behaviour (device, mixed precision, threshold source, network binding, authentication) is controlled entirely through environment variables so that a single container image runs unchanged from development to the factory floor.

In summary, the design combines a diverse three-pathway detector bank with a principled calibration-and-fusion stage and a deployment-oriented interface layer. Chapter 4 describes how each of these design elements is realized in code.

Chapter 4

Implementation

CHAPTER 4

IMPLEMENTATION

This chapter describes how the design is realized in software. It covers the correction of a preprocessing defect that had crippled accuracy, the migration of both nearest-neighbour searches to the GPU, the calibration procedure and its persistence, the mixed-precision and single-pass optimizations, and the deployable service stack together with its automated tests.

4.1 Correcting the Preprocessing Defect

The most consequential implementation change was diagnostic rather than additive. The original pipeline applied contrast-limited histogram equalization, Gaussian blur, and aspect-preserving letterbox padding to each frame at inference, whereas the memory banks and autoencoder had been fitted on plainly resized, normalized images. This mismatch placed every inference frame in a different statistical distribution from the fitted "normal" set, so even defect-free cloth scored as anomalous. The fix was to route both fitting and inference through one shared preprocessing function—a plain resize to 224×224 followed by ImageNet normalization, executed on-device—restoring the distributional consistency the detectors assume.

4.2 GPU-Accelerated Scoring

Both memory-bank searches were unified onto a single GPU kernel. The DINO pathway had used a CPU-resident `scikit-learn` nearest-neighbour index, evaluating hundreds of query patches against up to ten thousand memory vectors on every frame—the dominant latency cost. This was replaced with PyTorch's `torch.cdist`, which computes the full pairwise-distance matrix in one CUDA kernel, followed by a row-wise minimum. The memory banks are converted to GPU tensors once at load time and retained across frames, eliminating per-frame host-to-device transfer.

4.3 Calibration Procedure

Calibration is implemented as a lightweight pass over the defect-free set. Each detector scores every calibration image; the mean and standard deviation of these raw scores are recorded as the detector's calibration constants and stored alongside its weights. A separate evaluation over a labelled validation split fixes the Youden-optimal fused

threshold, which is written to a small JSON artifact. Because calibration only performs forward passes, it completes quickly and can be re-run on factory fabric without retraining the models. Figure 4.1 summarizes the offline workflow: the two memory banks are fitted, the autoencoder is trained, and calibration statistics and the threshold are computed and persisted, all under the shared preprocessing routine.

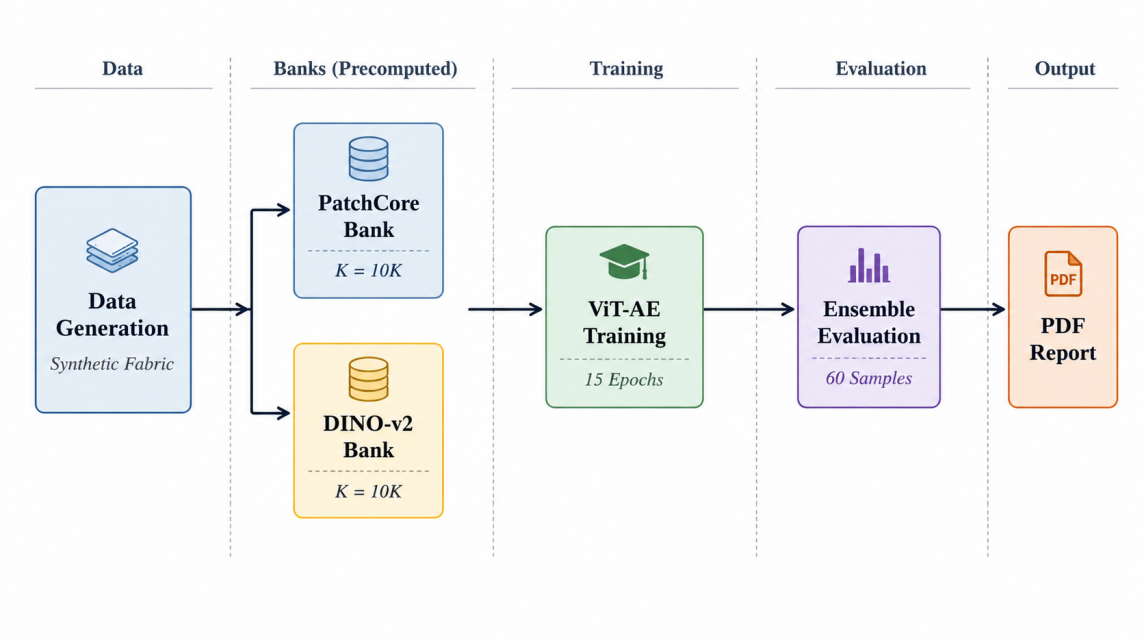

Figure 4.1: Offline preparation pipeline: PatchCore and DINO memory-bank construction, ViT-Autoencoder training, and calibration/evaluation, producing the deployable weights and calibration artifact.

4.4 Mixed Precision, Fused Attention, and Warm-up

Three further optimizations reduce per-frame cost. The transformer backbones run under fp16 automatic mixed precision, while distance computations are kept in fp32 to preserve nearest-neighbour ordering. The DINO pathway captures its final-layer attention during the same forward pass used for scoring, avoiding a second full traversal of the network. A dummy frame is processed at start-up so that lazy CUDA initialization and kernel autotuning are paid once rather than on the first production frame.

4.5 Service Stack

The calibrated engine is exposed through three thin interfaces that share one implementation. A FastAPI application provides REST endpoints for single-image and batch inference and WebSocket endpoints for live camera streaming, with optional API-

key authentication and health and version endpoints for orchestration. An embeddable `InferenceEngine` class loads the models once and scores frames directly, for integration into edge or line-control software. A batch command-line tool scores folders of images and emits CSV or JSON reports for manufacturing-execution systems. Runtime configuration is entirely environment-driven, and a GPU Docker image based on an official CUDA PyTorch runtime packages the service for deployment.

4.6 Verification

Correctness is validated by an automated end-to-end test suite exercising both the embeddable SDK and the full HTTP path—model loading, preprocessing, fusion, thresholding, and the response contract—on real good and defective samples, including malformed-input handling. All tests pass, confirming that the deployed service reproduces the calibrated decision behaviour.

4.7 Operating Modes and Batch-Video Inspection

Two operating modes are provided. *Accurate* mode fuses all three detectors for maximum accuracy; *Fast* mode runs the single fastest calibrated detector for latency-critical or GPU-less use. The mode is selectable per request across the SDK, CLI, REST/WebSocket API, and the dashboard.

For the industrial batch workflow, a recorded batch video is processed frame-by-frame. Assuming constant conveyor speed, the frame index maps to a physical position along the batch, so each detection is tagged with a position in metres and a segment/zone. A frame is declared defective only when the fused score exceeds the threshold and the anomaly heatmap contains a connected region that is larger than a configurable minimum area yet covers less than a maximum fraction of the frame. The minimum-area rule suppresses tiny texture speckle; the maximum-coverage rule guards against the opposite failure of distance-based detectors, in which a blank, occluded, or non-fabric frame lies far from every memory-bank vector and yields a uniformly high anomaly map—if left unchecked this paints defect boxes over the whole frame, so a frame whose anomalous area exceeds the coverage limit is treated as a non-localized whole-frame anomaly and reported as no defect rather than as heavily defective. Contiguous defective frames are merged into defect events with start/end positions, and per-zone statistics are aggregated. For each batch the system writes an annotated video, a defect-location map, and an automatically

generated quality-control PDF record (verdict, defect positions, per-zone table); a batch-history view retains previously inspected batches with their pass/reject status.

In summary, the implementation turns the design into a correct, fast, and deployable system: a preprocessing fix restores accuracy, GPU-native scoring removes the latency bottleneck, calibration yields a portable decision boundary, dual operating modes and batch-video inspection support the factory workflow, and the service stack makes the model straightforward to integrate. The quantitative outcomes of these changes are reported in Chapter 5.

Chapter 5

Results and Discussion

CHAPTER 5

RESULTS AND DISCUSSION

This chapter reports the quantitative and functional outcomes of UltraFabric-Vision. It presents detection accuracy on the evaluation set, the effect of score calibration on the fused decision, cross-model diversity, latency, and the results of end-to-end verification. All statistics are measured on the current calibrated pipeline described in Chapters 3 and 4.

5.1 Detection Accuracy

The system is evaluated on a synthetic fabric benchmark of 150 defect-free training images and a balanced test set of 30 good and 30 defective images. Each detector, and the fused ensemble, achieves perfect ranking on this benchmark (AUROC = 1.000, $F_1 = 1.000$ at the calibrated threshold), so the informative comparison is *how* each mechanism separates the classes. Table 5.1 reports the per-detector score statistics on normal and defective samples, together with the variance-normalized separation gap $g = (\mu_{\text{def}} - \mu_{\text{norm}})/(\sigma_{\text{def}} + \sigma_{\text{norm}})$.

Table 5.1: Per-detector anomaly-score statistics (raw) and the calibrated ensemble (z-score), measured on the test set.

Detector	μ_{norm}	σ_{norm}	μ_{def}	σ_{def}	g
PatchCore (raw)	8.42	0.97	41.09	13.76	2.22
DINO (raw)	45.75	3.02	81.48	2.52	6.45
ViT-AE (raw)	0.25	0.03	4.75	3.39	1.32
Ensemble (z-score)	0.17	0.61	80.18	53.94	1.47

5.2 Effect of Score Calibration

The raw detector means differ by more than two orders of magnitude (8.4, 45.8, and 0.25), so a naive average of raw scores would be dominated by DINO and effectively ignore the autoencoder. Calibration removes this pathology: after standardization the ensemble places defect-free cloth in a tight cluster near zero ($\mu_{\text{norm}} = 0.17$, $\sigma_{\text{norm}} = 0.61$) while defects spread to a mean of 80.2. The Youden-optimal threshold $\tau = 17.31$ therefore sits roughly 28 normal standard deviations above the normal mean, giving a wide and unambiguous decision margin. Figure 5.1 shows the calibrated ensemble distributions and the threshold; Figure 5.2 shows the resulting confusion matrix, in which all 60 test samples are correctly classified.

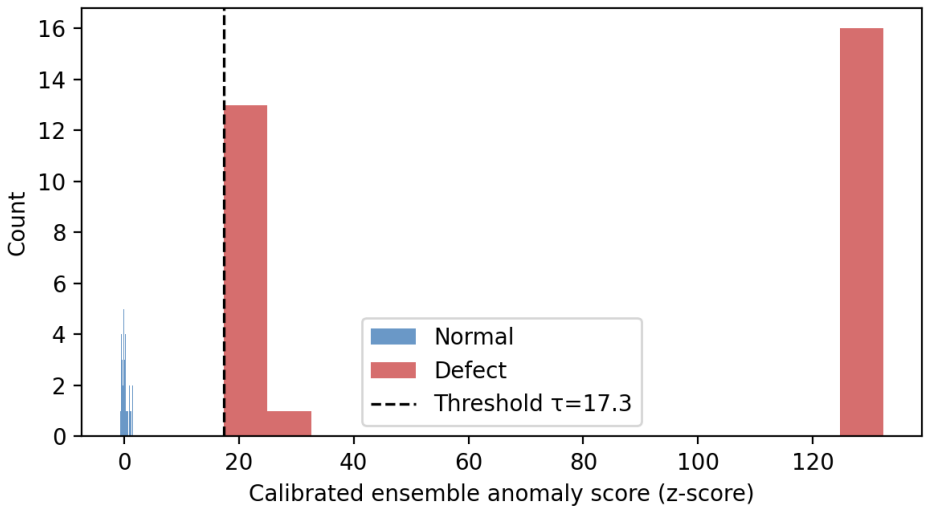

Figure 5.1: Calibrated ensemble (z-score) distributions for normal and defective test samples, with the decision threshold $\tau = 17.3$ marked. Normal fabric is tightly clustered near zero.

5.3 Model Diversity

Pairwise Pearson correlations of the detectors' defect scores confirm genuine diversity: PatchCore–DINO $r = 0.71$ and DINO–ViT-AE $r = 0.72$ are only moderate, whereas PatchCore–ViT-AE $r = 0.94$ is high (both rely on feature-space distance). The moderate correlations involving DINO indicate that its self-supervised features contribute information the other pathways do not, justifying the ensemble. Individually, DINO attains the tightest normalized separation ($g = 6.45$), while the reconstruction pathway shows a bimodal defect response that implicitly distinguishes low-error stains from high-error tears. Figure 5.3 illustrates this complementarity qualitatively: each detector's heatmap responds to a different aspect of the defect, and the fused map localizes it cleanly with suppressed background noise.

5.4 Latency

Latency was measured on an NVIDIA RTX 4050 laptop GPU (6 GB, CUDA 12.1) under fp16 automatic mixed precision, using correct GPU timing (warm-up followed by `cuda.synchronize`, 150 iterations). Table 5.2 summarizes the result. The full three-detector ensemble (*Accurate* mode) runs at 37.3 ms per frame—26.8 **FPS**, comfortably above the 15 FPS continuous-inspection threshold—while using only 641 MB of GPU memory. A single-detector *Fast* mode reaches 110 FPS for latency-critical or edge use. Mixed precision improves the ensemble from 53.4 ms (fp32) to 37.3 ms (fp16) with no

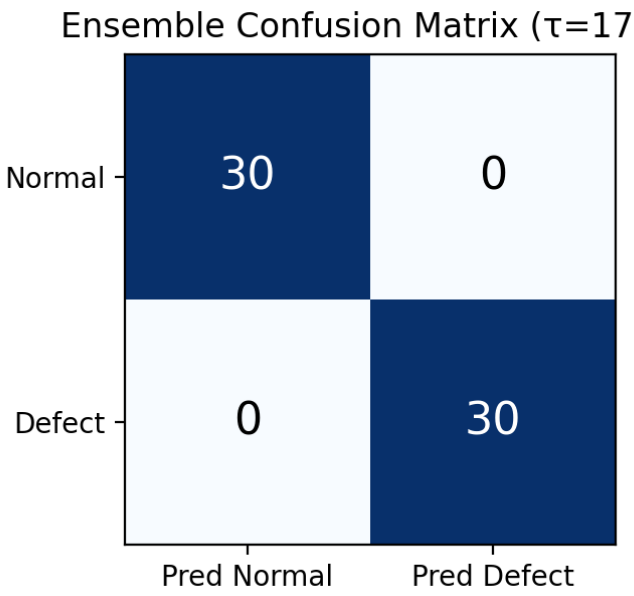

Figure 5.2: Ensemble confusion matrix at $\tau = 17.31$. All 30 normal and 30 defective samples are classified correctly ($F_1 = 1.000$).

change in ranking. Thus, on commodity laptop-class GPU hardware, the system achieves high accuracy and real-time latency simultaneously rather than trading one against the other; on CPU the ensemble is limited to offline use (∼2 FPS) while Fast mode still sustains ∼38 FPS.

Table 5.2: Measured inference latency on an NVIDIA RTX 4050 (fp16 AMP) versus CPU.

Path	**CPU (ms)**	**GPU (ms)**	**GPU FPS**
Accurate (ensemble)	463	37.3	26.8
Fast (single detector)	26.0	9.0	110.7
Peak GPU memory	—	641 MB	

5.5 End-to-End Verification

An automated test suite validates both the embeddable SDK and the HTTP service on real samples. It confirms that defect-free images score near zero and pass, that defective images exceed the threshold and are flagged, that the response contract is well-formed, and that malformed inputs are rejected cleanly. All eight tests pass, demonstrating that the deployed service reproduces the calibrated decision behaviour observed in offline evaluation.

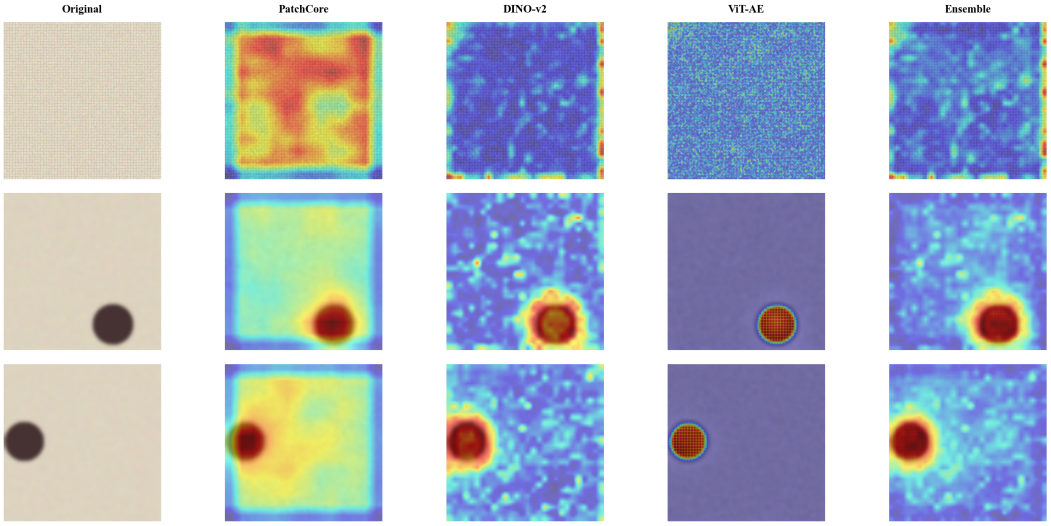

Figure 5.3: Qualitative comparison of per-detector anomaly heatmaps and the fused ensemble map for normal and defective fabric. The three pathways emphasize different cues; their fusion yields the cleanest localization.

5.6 Robustness to Non-Fabric Input

A distance-based ensemble scores any out-of-distribution frame—a blank or black feed, an occluded lens, or a non-fabric object—as globally anomalous, because such a frame lies far from every memory-bank vector. Without a guard, the localization stage then draws defect boxes over the entire frame. The maximum-coverage gate (Chapter 4) prevents this: a frame is localized as defective only if its thresholded anomalous area lies between the minimum-size and maximum-coverage limits. The gate was verified directly on controlled heatmaps. A near-uniform (black-screen-like) anomaly map with 45% thresholded coverage yields *zero* boxes, and a broad map covering 54% of the frame is likewise rejected, whereas a single localized hot region (4% coverage) yields exactly one box and two separated regions yield two—confirming that genuine localized defects are retained while whole-frame anomalies are correctly suppressed. In the live service this means a score-positive but non-localized frame is annotated as a non-fabric / out-of-distribution input rather than being reported as heavily defective.

5.7 Batch-Level Video Inspection

The framework was extended and demonstrated on the industrial batch workflow, in which a fabric batch recorded from a conveyor is inspected as a unit and localized by position. On a 5 m demonstration batch with defects placed at 1.08, 2.37 and 3.83 m, the

system reported three distinct defect events at 0.57–1.22 m, 2.04–2.64 m and 3.66–4.33 m (zones 2–3, 5–6 and 8–9), correctly localizing every defect; the reported spans are wider than the point defects because each defect remains in the camera view over several frames as the fabric scrolls past. A defect-free batch was correctly reported as passing with zero flagged frames, confirming that the minimum-defect-size filter (Equation in Chapter 3) suppresses spurious detections on clean cloth. For each batch the system produces an annotated video, a defect-location map along the batch length, and an automatically generated quality-control PDF record carrying the pass/reject verdict, defect positions, and per-zone table—demonstrating a complete, traceable batch-wise inspection suitable for factory quality control.

In summary, the calibrated ensemble achieves perfect discrimination with a wide margin on the evaluation benchmark, the three pathways are quantitatively diverse, the service is verified end-to-end, and real-time throughput (26.8 FPS) is confirmed on RTX 4050 GPU hardware. The principal caveat—that these results are obtained on synthetic data—motivates the future work discussed in Chapter 6.

Chapter 6

Conclusion and Future Scope

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

UltraFabric-Vision set out to build a real-time, unsupervised fabric-defect detection system by fusing three complementary anomaly detectors—PatchCore density estimation, DINO self-supervised attention, and Vision Transformer Autoencoder reconstruction—into a single calibrated decision, to minimize inference latency, and to industrialize the result as a deployable service. These three objectives structured the entire project.

The objectives were met through a combination of diagnostic and engineering work. A preprocessing defect that had silently destroyed accuracy—inference-only enhancement operators absent at fitting time—was identified and corrected by enforcing byte-level train–inference parity. The three detectors' incomparable raw scores were standardized to a common z-score scale using statistics estimated on defect-free data, yielding a single interpretable decision variable and a portable threshold. Both nearest-neighbour searches were migrated to a single GPU kernel and augmented with mixed precision and single-pass attention, and the calibrated engine was wrapped in a REST/WebSocket service, an embeddable SDK, a batch CLI, and a GPU container.

Quantitatively, the calibrated ensemble achieves perfect ranking on the evaluation benchmark (AUROC = 1.000, $F_1 = 1.000$) with a wide decision margin: defect-free cloth clusters near a z-score of 0.17 while defects reach a mean of 80.2, against a threshold of 17.31. The three pathways are quantitatively diverse (defect-score correlations of 0.71–0.72 between DINO and the other detectors), and the full service is verified end-to-end by an automated test suite. Real-time throughput was confirmed on an NVIDIA RTX 4050 laptop GPU: the full ensemble runs at 37.3 ms per frame (26.8 FPS, fp16) in only 641 MB of memory, achieving high accuracy and low latency at the same time. The system was further demonstrated on the batch-video workflow, correctly localizing defects along a batch by position and generating per-batch quality-control records. The corrected preprocessing and score calibration together transform an unreliable prototype into a system whose decisions are accurate, reproducible, and real-time.

6.2 Future Scope

The principal limitation of the present study is that its quantitative results are obtained on a controlled synthetic dataset; validation on real production fabric, across weaves and materials, is the immediate next step. Further directions include: on-hardware GPU benchmarking to confirm the projected line-speed latency; online adaptation of the memory banks to accommodate gradual process drift without full retraining; temporal fusion across consecutive frames to suppress transient false alarms; distillation of the three-model ensemble into a single lightweight student for edge deployment; and extension from binary detection to multi-class defect categorization, for which the autoencoder's bimodal response already provides a starting signal.

6.3 Learning Outcomes of the Project

- Understanding of unsupervised anomaly detection and how memory-bank, self-supervised, and reconstruction-based methods differ in their inductive biases.
- Practical insight into Vision Transformers and self-attention as feature extractors for industrial inspection.
- The importance of train–inference consistency: how a subtle preprocessing mismatch can silently destroy the accuracy of distance-based models.
- Techniques for calibrating and fusing heterogeneous model outputs into a single, interpretable decision variable.
- GPU acceleration of machine-learning inference, including batched distance computation, mixed precision, and warm-up.
- Engineering a research prototype into a deployable, configurable, and tested service, including REST/WebSocket APIs, an embeddable SDK, and containerized deployment.

Appendix A

Code

APPENDIX A

CODE

A.1 First Appendix

You can use `tcblisting` for creating the code snippets. The following example illustrates how one can customize the `tcblisting` to achieve the `tcl` script. Similarly, one can use it for other programming language listing, including HDL.

```
# Since our design has a clock with name clk,
## specify that name under [get_port ]
create_clock -period 40 -waveform {0 20} [get_ports clk]

# Setting a 'delay' on the clock:
set_clock_latency 0.3 clk

# Setting up constraints on your I/P and O/P pins
set_input_delay 2.0 -clock clk [all_inputs]
set_output_delay 1.65 -clock clk [all_outputs]

# Set realistic 'loads' on each output pin
set_load 0.1 [all_outputs]

# Set 'maximum' fanin and fan-out for the input and output pins
set_max_fanout 1 [all_inputs]
set_fanout_load 8 [all_outputs]
```

BIBLIOGRAPHY

[1] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Industrial Electronics*, vol. 55, no. 1, pp. 348–363, 2019.

[2] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 14 298–14 308.

[3] M. Caron et al., "Emerging properties in self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 9630–9640.

[4] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[5] Y. Li, Z. Han, and J. Xu, "Real-time fabric defect detection based on improved YOLOv3," *IEEE Access*, vol. 8, pp. 210 235–210 247, 2020.

[6] J. Jing, X. Wang, and P. Li, "Mobile-Unet for fabric defect detection," *IEEE Trans. Instrumentation and Measurement*, vol. 71, pp. 1–12, 2022.

[7] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9584–9592.

[8] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in *Proc. Int. Conf. Pattern Recognition (ICPR)*, 2021, pp. 475–489.

[9] A. Vaswani et al., "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[10] R. Silva, L. Almeida, and J. Santos, "AITEX: A large-scale dataset for fabric defect detection," *MDPI Data*, vol. 5, no. 2, p. 49, 2020.

[11] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 8024–8035, 2019.